Mohammad Abu Shariha CMPSC 465 HW8 **Problem 1, Part (a)**

(a) Kruskal's algorithm — order of edges added (ties broken lexicographically)

First, we list the edges sorted by weight, breaking ties lexicographically:

- (c, e): 3
- (a, d): 4
- (b, f): 4
- (c, f): 5
- (e, f): 6
- (a, c): 7
- (b, c): 7
- (a, g): 8
- (b, g): 8
- (d, e): 9

Kruskal's algorithm adds edges from this list as long as they do not form a cycle. We need $7-1=6$ edges for the MST.

1. **Add (c, e)**, w=3. Components: {a}, {b}, {d}, {f}, {g}, {c, e}
2. **Add (a, d)**, w=4. Components: {b}, {f}, {g}, {c, e}, {a, d}
3. **Add (b, f)**, w=4. Components: {g}, {c, e}, {a, d}, {b, f}
4. **Add (c, f)**, w=5. (Connects {c, e} and {b, f}). Components: {g}, {a, d}, {b, c, e, f}
5. **Skip (e, f)**, w=6. (Both 'e' and 'f' are in the same component, this would form a cycle).
6. **Add (a, c)**, w=7. (Connects {a, d} and {b, c, e, f}). Components: {g}, {a, b, c, d, e, f}
7. **Skip (b, c)**, w=7. (Both 'b' and 'c' are in the same component, this would form a cycle).
8. **Add (a, g)**, w=8. (Connects {g} and the main component). Components: {a, b, c, d, e, f, g}

We now have 6 edges and all vertices are in one component.

So the Kruskal edge sequence is: **(c, e), (a, d), (b, f), (c, f), (a, c), (a, g)**. These 6 edges form the MST. Total weight: $3+4+4+5+7+8=31$.

Problem 1, Part (b)

(b) Prim's algorithm - order of vertices added

We assume Prim starts at the lexicographically smallest vertex **a**. At each step choose the minimum-weight edge from the tree to a new vertex.

Step-by-step:

- Start: add **a**. (Order so far: a)
- Frontier edges from a: (a,d)=4, (a,c)=7, (a,g)=8 → pick (a,d)=4 → add **d**. (a, d)
- From {a,d}: frontier (a,c)=7, (a,g)=8, (d,e)=9 → pick (a,c)=7 → add **c**. (a, d, c)
- From {a,d,c}: frontier includes (c,e)=3, (c,f)=5, (b,c)=7, ... → pick (c,e)=3 → add **e**. (a, d, c, e)
- Frontier now has (c,f)=5, (e,f)=6, (b,c)=7, ... → pick (c,f)=5 → add **f**. (a, d, c, e, f)
- Frontier edges to new vertices include (b,f)=4 — note its weight 4 is the smallest available → add **b** next. (a, d, c, e, f, b)
- Only **g** remains; the smallest edges to g are (a,g)=8 and (b,g)=8. We add **g** (via (a,g)=8).

Order of vertices added by Prim starting at a: **a, d, c, e, f, b, g**.

Problem 2

Problem: Given undirected graph $G = (V, E)$ and integer k, find the largest vertex set $V' \subseteq V$ (and corresponding induced subgraph) such that every vertex in V' has degree at least k *within that subgraph.* Want algorithm in $O(|V| + |E|)$.

algorithm (peeling/ iterative removal)

1. Compute degree $\deg[v]$ for every vertex v. Put all vertices with $\deg[v] < k$ into a queue Q.
2. While Q not empty: pop vertex u. If u already removed, continue. Mark u removed (not in V'). For each neighbor w of u that is not removed, decrement $\deg[w]$ by 1; if now $\deg[w] < k$, push w into Q.
3. After the queue empties, the remaining (not-removed) vertices form the desired subgraph $G' = (V', E')$. Return G'.

Correctness (sketch)

Any vertex with degree $< k$ cannot belong to any k-degree subgraph, so it must be removed. Removing those vertices can reduce neighbors' degrees below k, which must also be removed, and so on. This process removes exactly the vertices that cannot appear in any subgraph where all vertices have degree $\geq k$. what remains is maximal (contains all vertices that could possibly remain) and every remaining vertex has degree $\geq k$ (because otherwise it would have been queued and removed). Hence the result is a largest possible such subgraph.

Runtime

Each vertex enters queue at most once and each edge is examined at most twice (once per endpoint) during neighbor degree updates. so runtime is $O(|V| + |E|)$. Space $O(|V| + |E|)$.

Problem 3, Part (a)

(a) Use DSU to check whether mandatory edges form a cycle

Initialize a Disjoint Set Union (Union-Find) structure with all vertices as singleton sets. For each **mandatory** edge (u, v) do:

- If find(u) == find(v) then adding this mandatory edge would create a cycle $\rightarrow$ mandatory edges contain a cycle $\rightarrow$ no feasible spanning tree exists; stop.
- Else union(u, v).

If you process all mandatory edges without detecting find(u) == find(v), the mandatory edges are cycle free.

this is correct because a spanning tree is acyclic; any cycle among mandatory edges makes it impossible to include all of them in any spanning tree.

Running time: $O(|V| + m\alpha(|V|))$ where m is number of mandatory edges (practically linear).

Problem 3, Part (b)

(b) Modify Kruskal to construct the minimum-cost feasible spanning tree (if any)

Algorithm

1. Use DSU and process all **mandatory** edges first:
   - If any mandatory edge creates a cycle (find(u) == find(v)), return "no feasible spanning tree" (impossible).
   - Else union their endpoints and add those edges to the partial spanning forest.
2. Build a list of all **optional** edges (ignore forbidden edges entirely). Sort optional edges by increasing weight.
3. Run Kruskal over the optional edges but with the DSU already containing the unions of mandatory edges: for each optional edge (u, v) in increasing order, if find(u) $\neq$ find(v) then union(u, v) and add the edge.
4. After processing all optional edges, check if the DSU has exactly one component (or you have $n-1$ total edges including mandatory ones). If yes $\rightarrow$ you have a spanning tree that includes all mandatory and excludes all forbidden edges; the edges chosen form the **minimum-cost feasible spanning tree**. If not $\rightarrow$ **no feasible spanning tree** (the mandatory + optional edges are not sufficient to connect all vertices).

Correctness

Any feasible spanning tree must include all mandatory edges and no forbidden edges. After forcing mandatory edges and then adding optional edges in increasing weight order while avoiding cycles, you follow the same greedy correctness argument as Kruskal restricted to the set of allowed edges; since Kruskal produces the MST on any set of allowed edges, this yields the minimum-cost spanning tree among all spanning trees that contain all mandatory edges and exclude forbidden edges (if one exists).

Runtime

Sorting optional edges ($O(|E| \log |E|)$). DSU operations overall ($O(|E|\alpha(|V|))$). So total $O(|E| \log |E|)$.

Problem 4, Part (a)

(a) Prove or disprove: T remains an MST under w'.

Proof (positive)

For any spanning tree S with original total weight $W(S) = \sum_{e \in S} w(e)$, the new total is

$$W'(S) = \sum_{e \in S} (a \cdot w(e) + b) = a \cdot W(S) + (n-1) \cdot b,$$

since every spanning tree has exactly $n-1$ edges. Because $a > 0$, the function $W(S) \mapsto a \cdot W(S) + (n-1) \cdot b$ is strictly increasing in $W(S)$. So the tree T that minimizes $W(S)$ also minimizes $W'(S)$. Thus T remains an MST under w'.

(Notice $b \geq 0$ is irrelevant for this argument; any constant b added to every edge contributes the same additive offset $(n-1)b$ to every tree.)

Problem 4, Part (b)

(b) Generalize to $a > 0$ and b any real (possibly negative).

Generalization

The same argument works for any real b. For any real b,

$$W'(S) = a \cdot W(S) + (n-1) \cdot b.$$

Because $a > 0$, the mapping is strictly increasing in $W(S)$. Therefore the tree minimizing $W(S)$ still minimizes $W'(S)$. So for any $b \in \mathbb{R}$ (negative or positive), as long as $a > 0$, the original MST remains an MST under the transformed weights.

Problem 5, Part (a)

(a) New edge $e' = (u, v)$ with weight $w'(e')$ is added. How to decide efficiently whether T remains the MST?

Key fact (cut-cycle property)

Inserting an edge $e' = (u, v)$ creates exactly one cycle if you add it to T (the unique path $P_T(u, v)$ in T plus e'). Let $e_{\max}$ be the edge on that path with the **largest** weight. If $w'(e') < w(e_{\max})$ then replacing $e_{\max}$ by e' yields a strictly smaller spanning-tree weight $\rightarrow T$ is no longer the MST. If $w'(e') > w(e_{\max})$ then T remains the MST. (With distinct weights there is no equality case.)

Algorithmic approaches

- **Naïve:** find the unique path from u to v in T and compute the maximum-weight edge on that path. This costs $O(|V|)$ worst-case.
- **Fast:** preprocess T for maximum-edge-on-path queries (e.g., with Heavy-Light Decomposition) in $O(|V| \log |V|)$ preprocessing; then each query can be answered in $O(\log |V|)$.

Complexity

Naïve single check without preprocessing: $O(|V|)$.

Problem 5, Part (b)

(b) Edge weight increase: an existing edge e has its weight increased from $w(e)$ to $w'(e) > w(e)$. How does MST change?

Two cases:

1. **If** $e \notin T$: increasing a non-tree edge cannot make T invalid as an MST. So T **remains the MST**.
2. **If** $e \in T$: increasing the weight may make e no longer part of the MST. Remove e from T; this splits T into two components A and B. Among all edges in the original graph G that connect A and B (i.e., crossing edges of that cut), the MST must contain the minimum-weight such edge. So:
   - Find the minimum-weight edge e_{best} (over all edges in $E \setminus \{e\}$) that connects the two components A and B.
   - If $w'(e) \leq w(e_{\text{best}})$ then even after the increase e remains the lightest crossing edge and T **remains the MST**.
   - If $w'(e) > w(e_{\text{best}})$ then we must replace e with e_{best} to get the new MST.

How to find e_{best} efficiently

- **Naïve:** check all edges in E to see which cross the cut ($O(|E|)$).
- **Better:** maintain dynamic tree/cut data structures (e.g., link-cut trees) to support updates/queries in $O(\log |V|)$ amortized time.

Summary complexity (simple approach)

If using a full scan of edges after removing e: $O(|E|)$ to find e_{best}.